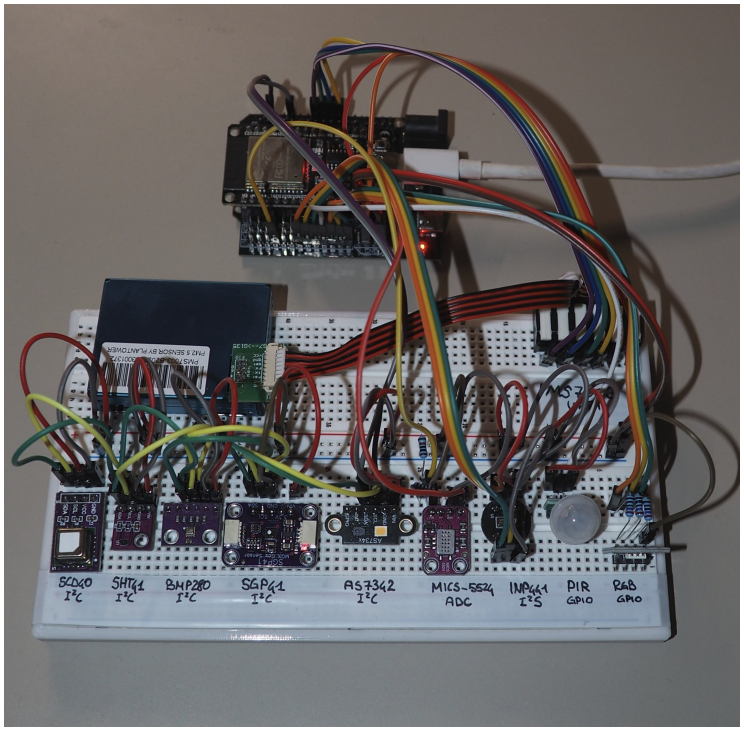
Bern University of Applied Sciences
Engineering and Computer Science
Electrical Engineering and Information Technology

RTOS Project Work

Electrical Outlet IoT

Environmental monitoring node on ESP32 with FreeRTOS and MQTT



Breadboard prototype of the sensor node.

Authors: Simone Pelascini | Aurélien Bollin

Date: April 26, 2026

Contents

1 Summary

The **Electrical Outlet IoT** project is an ESP32-based environmental monitoring node designed to be integrated into a standard wall-outlet form factor. The motivation behind this project is the growing demand for **ambient air quality monitoring** and **occupancy-aware automation** in residential and commercial buildings. Poor indoor air quality, driven by CO_2 accumulation, volatile organic compounds (VOC), particulate matter (PM), and combustible gas, is a largely invisible health risk. Existing commercial solutions are either too expensive, closed-source, or limited to a single measurement type.

This project addresses that gap by combining nine sensors on a single node, connected to a local MQTT broker for real-time telemetry. The firmware is built on *PlatformIO*, *ESP-IDF*, and *FreeRTOS*, making use of four concurrent tasks that coordinate through a **mutex-protected global state structure**. Sensor data is published as JSON payloads and consumed by a companion Node.js dashboard that renders live charts via *Chart.js* and pushes updates to browsers via *Socket.IO*.

The current repository documents breadboard bring-up, firmware architecture, runtime characterisation, and the web-dashboard edge. The form-factor integration into an actual outlet housing is left as a future step.

2 System Context

2.1 Hardware

The sensor node is built around an **ESP32 dual-core 240 MHz** microcontroller (Espressif `esp32dev` target). It communicates with nine sensors over five distinct interfaces. Table 1 summarises the complete sensor inventory.

Table 1: Complete sensor inventory used by the ESP32 node.

Sensor	Measured quantity	Physical value	Bus	Driver
SCD40	CO_2, T, RH	Concentration, temperature, humidity	I^2C	Direct ESP-IDF
SHT41	Temperature, humidity	High-precision T & RH	I^2C	esp-idf-lib
SGP41	VOC, NOx	Gas indices 0–500 (Sensirion algorithm)	I^2C	Custom + GIA
BMP280	Pressure, temperature	Atmospheric P & T	I^2C	esp-idf-lib
AS7341	Light spectrum	8-channel F1–F8 basic counts	I^2C	k0i05/esp_as7341
PMS7003	Particulate matter	PM1.0, PM2.5, PM10 [µg/m^3]	UART	Custom wrapper
MiCS-5524	Combustible gas	Voltage [V], heuristic CO [ppm]	ADC	Custom
AS312	PIR motion	Motion detected (boolean)	GPIO	Custom
INMP441	Ambient noise	Sound pressure level [dB SPL]	I^2S	Custom DSP

2.2 System actors and data paths

The system involves the following actors:

- **Sensor node**: ESP32 (`esp32dev`), firmware in `src/` and `lib/`. Samples all sensors on software-defined deadlines, aggregates readings into a shared in-memory state, and transmits them via Wi-Fi.
- **Wi-Fi / TCP**: the node connects as a Wi-Fi station; all MQTT traffic flows over a standard TCP connection to the broker.
- **MQTT broker**: any standard broker (e.g. Mosquitto) reachable on the LAN. Credentials and URI are configured in `lib/config/include/app_config.h`.
- **Node.js dashboard**: an optional `server/` application that subscribes to all device topics and renders live charts in the browser via Socket.IO and Chart.js.
- **Third-party consumers**: any MQTT subscriber can access the JSON payloads independently of the dashboard.

2.3 MQTT topics

All publications follow the hierarchy `devices/<device_id>/` where `device_id` is set by `APP_DEVICE_ID`:

- `telemetry/environment` (QoS 0) — full sensor snapshot: CO_2, T, RH from SCD40 and SHT41, atmospheric pressure, VOC/NOx indices, PM1.0/2.5/10, 8-channel light array, gas voltage and ppm, noise level in dB SPL. Published every **1 s**.
- `status/system` (QoS 1, retain) — system health flags: `system_ok`, `degraded_mode`, `wifi_connected`, `mqtt_connected`. Published every **10 s** and immediately on reconnect.
- `event/alarm` (QoS 1) — event-driven alert: `as312_alarm` (motion), `mics5524_alarm` (gas threshold exceeded), `co2_alarm_level` (IDA-inspired 6-level classification). Published only on the **rising edge** of each alarm flag.

2.4 Web dashboard

The companion dashboard runs on Node.js (`server/public/server.js`) and uses the `express`, `mqtt`, `socket.io`, and `dotenv` packages. It subscribes to the wildcard patterns `devices/+/telemetry/environment`, `devices/+/status/system`, and `devices/+/event/alarm`, merges incoming payloads into a per-device state object, and forwards it to all connected browsers in real time. Configuration is via a `.env` file (`MQTT_BROKER_URL`, optional credentials, `PORT`).

3 Requirements

Requirements are written from the user perspective and are traceable to implementation files. The **State** column uses colour coding: implemented, partial, or open.

Table 2: System requirements and current implementation status.

ID	Requirement (user perspective)	State
R1	The system shall boot, initialise all peripherals, create four FreeRTOS tasks, and reach a fully operational state without a fatal error within 20 s of power-on.	**implemented**
R2	The system shall connect to the configured Wi-Fi network within 15 s; on disconnect it shall retry up to `APP_WIFI_MAX_RETRY` times automatically.	**implemented**
R3	The system shall publish a JSON environment payload to the MQTT broker at least every 1 s and a system-status payload every 10 s when connected.	**implemented**
R4	The system shall publish an alarm event payload on the rising edge of a motion or gas alarm, without missing edges caused by task jitter.	**implemented**
R5	All concurrent accesses to `g_device_state` shall be serialised by a FreeRTOS mutex (`g_device_state_mutex`) with no unbounded blocking.	**implemented**
R6	All nine integrated sensors shall be polled within their specified deadlines without one slow sensor blocking another.	**implemented**
R7	The system shall report an estimated ambient noise level (`noise_db`) in dB SPL; absolute accuracy relative to a calibrated sound-level meter is not guaranteed without field calibration of `INMP441_SPL_OFFSET_DB`.	**partial**
R8	Stack high-water mark and heap usage shall be logged and reported for all tasks over at least 10 minutes of continuous operation.	**implemented**
R9	Automated unit tests shall cover MQTT payload builders and sensor supervision logic.	**open**
R10	The system shall attempt automatic sensor restoration (re-initialisation and recovery) after a detected sensor crash, with bounded retries and fault reporting if recovery fails.	**open**

4 Design

4.1 Boot sequence

At startup, `app_main` (in `src/main.c`) executes the following ordered steps. All critical steps abort the system on failure; non-critical sensor failures are logged as warnings and allow the system to continue in degraded mode.

1. **`device_state_init()`** — zero-initialises `g_device_state` and creates the two FreeRTOS mutexes (`g_device_state_mutex` and `g_sensor_driver_mutex`).
2. **`sensor_init_all()`** — initialises hardware peripherals in order: ADC unit 1, I^2S channel, I^2C bus, UART 2, then each sensor driver. Critical peripherals abort on failure; sensors warn and continue.
3. **`network_app_init()`** — initialises NVS, netif, Wi-Fi driver, and starts the connection; registers event handlers for `WIFI_EVENT` and `IP_EVENT`.
4. **`network_app_wait_until_connected(15000)`** — blocks up to 15 s waiting for `APP_WIFI_CONNECTED_BIT`; aborts if the bit is not set.
5. **`mqtt_app_start()`** — creates the ESP-IDF MQTT client, registers the event handler, and starts the client task.
6. **Task creation** — `xTaskCreate` is called for each of the four tasks; any allocation failure aborts immediately.

4.2 FreeRTOS task architecture

The firmware is decomposed into four independent FreeRTOS tasks. All tasks use `vTaskDelayUntil` for deterministic loop periods and `xSemaphoreTake` / `xSemaphoreGive` for exclusive access to the shared state. Table 3 summarises the static parameters.

Table 3: Static configuration of the four FreeRTOS tasks.

Task	Responsibility	Pri.	Stack (w)	Period
`acquisition_task`	Polls all sensors on individual software deadlines; writes measured values, validity flags, and fault flags into `g_device_state` under the mutex.	2	4096	1000 ms
`audio_task`	DMA-reads one audio block from the INMP441 via I^2S, computes AC-RMS SPL, and updates `noise_db` in the shared state.	2	4096	500 ms
`comm_task`	Snapshots the shared state and drives the MQTT publish schedule: environment telemetry at 1 s, system status at 10 s, and alarm events on rising edge.	3	4096	100 ms
`system_task`	Reads the shared state snapshot, evaluates per-sensor health checks (staleness, fault, validity) and alarm logic, then writes derived system flags back.	4	4096	100 ms

Design pattern — local state before commit. Each task maintains a task-local state structure initialised to zero. Sensor readings and flags are written to this local copy first; only when a consistent update is ready is the mutex acquired and the global `g_device_state` updated in a single block-copy. This minimises the critical section duration and avoids partially-updated state being observed by other tasks.

4.3 Sensor polling intervals

Sensors inside `acquisition_task` are polled on independent software timers derived from `xTaskGetTickCount()`. Each sensor deadline is checked at every 1000 ms task tick; the sensor is read only when `now - last_read` $\geq$ `interval`. This means a single slow sensor (e.g. PMS7003) does not delay faster ones (e.g. AS312).

Table 4: Sensor polling intervals and interfaces used by the firmware.

Sensor	Physical quantity	Interface	Interval
AS312	PIR motion	GPIO	1000 ms
MiCS-5524	Combustible gas (CO heuristic)	ADC	1000 ms
SGP41	VOC index, NOx index	I^2C	1000 ms
AS7341	8-channel light spectrum	I^2C	1000 ms
SHT41	Temperature, humidity	I^2C	2000 ms
BMP280	Atmospheric pressure	I^2C	2000 ms
SCD40	CO_2, temperature, humidity	I^2C	2500 ms
PMS7003	PM1.0, PM2.5, PM10	UART	5000 ms
INMP441	Noise level (dB SPL)	I^2S (audio_task)	500 ms

4.4 System supervision and alarm logic

`system_task` implements two independent modules:

Health module (`lib/health/`): after a 45 s cold-start window, each sensor is checked for three conditions: (1) the validity flag is `false`, (2) the fault flag is `true`, or (3) the timestamp age exceeds a per-sensor timeout (45 s for all sensors). Any failing check sets `degraded_mode = true` and the sensor-specific `degraded_*` flag.

Alarm module (`lib/alarm/`): three independent sub-checks evaluate (1) motion from the AS312 PIR, (2) gas concentration from MiCS-5524 against a 5000 ppm threshold, and (3) CO_2 from SCD40 classified into six levels (Optimal / Good / Moderate / Poor / Very Poor / Critical) following IDA and ASHRAE 62.1-inspired thresholds.

4.5 System context diagram

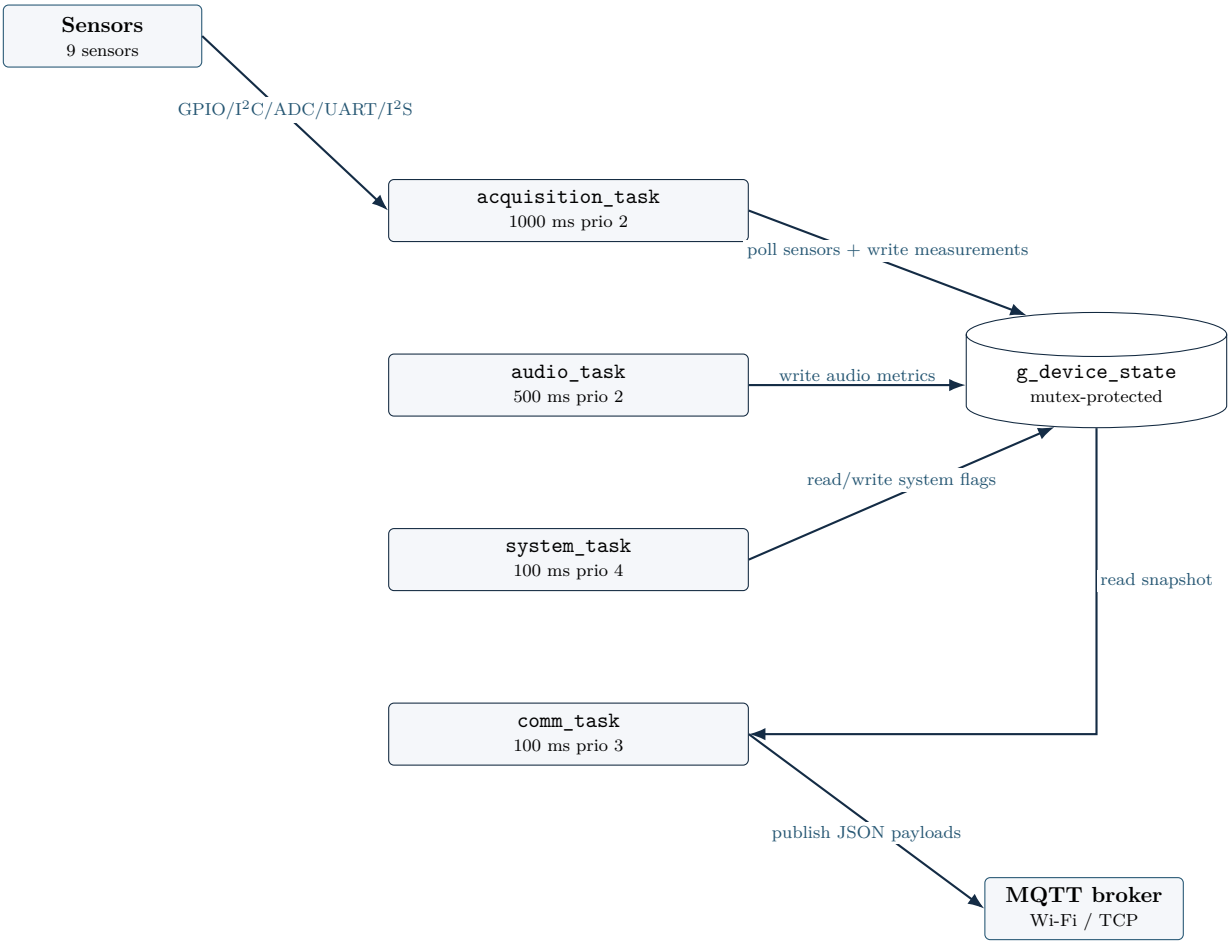


Figure 1: System context diagram showing sensors, FreeRTOS tasks, the mutex-protected shared state, and the MQTT broker used for publishing telemetry, status, and alarm messages.

4.6 UML task interaction diagram

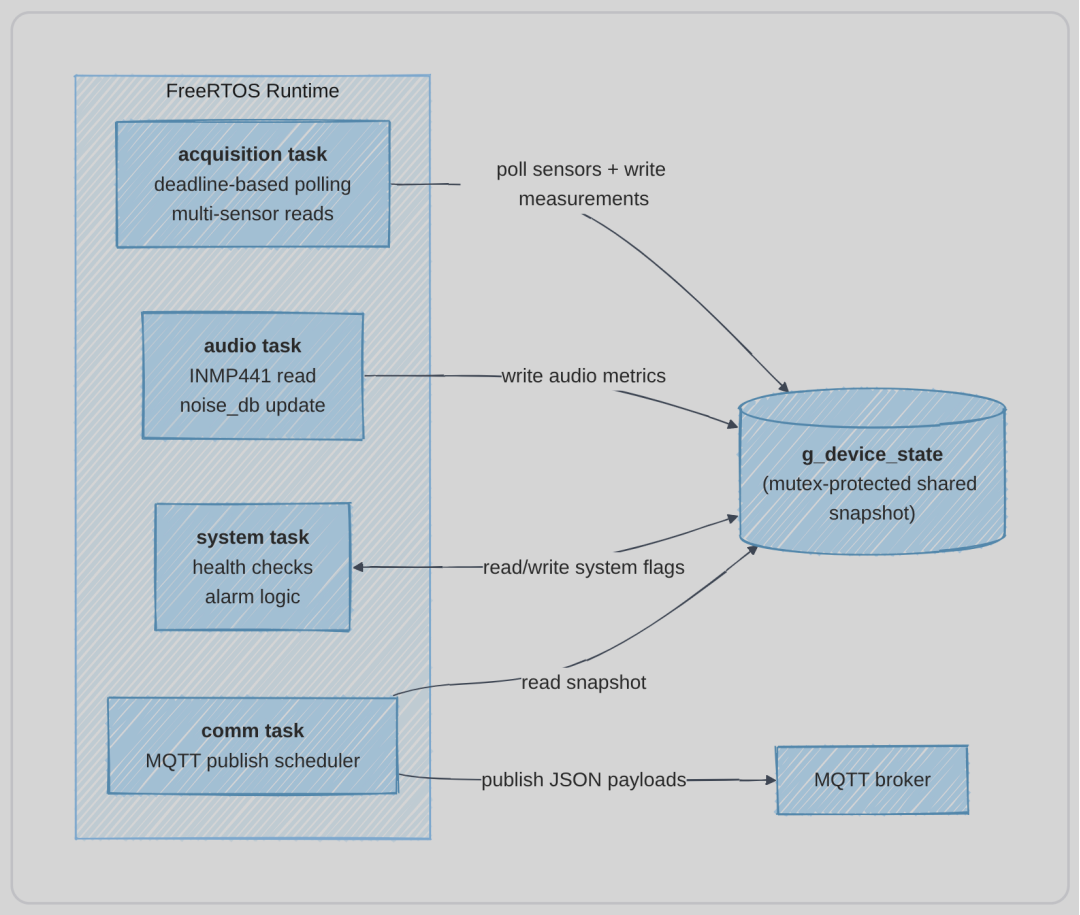


Figure 2: UML activity/sequence diagram of the four FreeRTOS tasks and their interactions with `g_device_state` and the MQTT client.

4.7 Acquisition polling flow

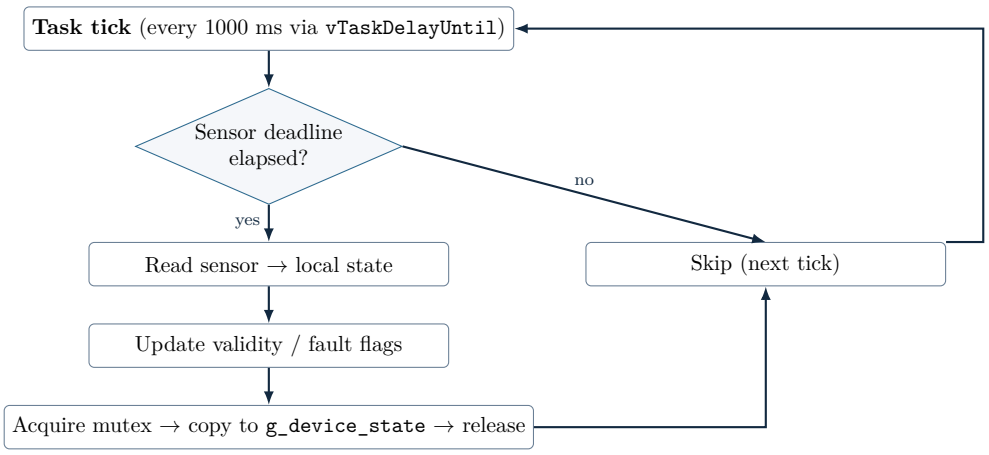


Figure 3: Deadline-driven polling inside `acquisition_task`: each sensor has its own independent timer; skipped sensors do not block the loop.

5 Runtime Statistics

5.1 Methodology

Runtime measurements were collected from the ESP32 serial log using the helper functions `logTaskStackHeapUsage()` and `logTaskAlive()`, defined in `src/task_config.c`. Each task logs its own stack high-water mark and the system-wide heap state every **10 s** of wall-clock time. The measurements span approximately **10 minutes** of continuous operation under normal conditions (all sensors active, Wi-Fi connected, MQTT publishing running). Raw data is archived in `raport/project_info/stack_heap.txt`.
Stack usage percentage is computed as:

$$\text{Stack Usage} = \frac{\text{words used (high-water mark)}}{\text{stack allocated (words)}} \times 100\%$$

where *words used* = allocated − high-water mark remaining (reported by `uxTaskGetStackHighWaterMark()`; 1 word = 4 bytes on ESP32).

5.2 Stack usage per task

Table 5: Measured stack usage per FreeRTOS task over the runtime window.

Task	Pri.	Stack alloc. (w)	Stack used (HWM, w)	Usage (%)	Avg. CPU (%)
`acquisition_task`	2	4096	2532	61.8%	n/m
`audio_task`	2	4096	2124	51.9%	n/m
`comm_task`	3	4096	2812	68.7%	n/m
`system_task`	4	4096	2352	57.4%	n/m

n/m = not measured (FreeRTOS runtime stats counter not enabled in this build).

Stack analysis. The highest consumer is `comm_task` (68.7%), followed by `acquisition_task` (61.8%). Even the worst case leaves **1284 words (5136 bytes) of free stack headroom** on `comm_task`, well above the ESP-IDF recommended minimum of 512 bytes. No stack overflow was observed during the measurement window. All four stacks are sized safely at the current feature set; if JSON payload building or additional sensors are added, `comm_task` should be monitored first.

5.3 Heap usage over time

The heap figures below are system-wide (MALLOC_CAP_8BIT pool, total ≈ 290 KB). "Min free" is the all-time minimum free heap reported by `heap_caps_get_minimum_free_size`, which is the most conservative safety margin.

Table 6: System heap usage at three measurement windows.

Window	Used (B)	Free (B)	Min free (B)	Usage	Notes
∼10 s	107 460	183 116	176 968	37.0%	Post-boot steady state
∼1 min	107 452	183 124	176 968	37.0%	Stable; marginal drift
∼10 min	107 432	183 144	176 968	37.0%	Stable baseline
Peak (transient)	108 992	181 584	176 968	37.5%	Acquisition spike

Heap analysis. Heap usage is highly stable at approximately **37%** of the total ≈290 KB pool, leaving a comfortable **63% free** (≈183 KB) during steady-state operation. The minimum-free-ever figure of **176 968 B** (≈173 KB) confirms that even transient allocation peaks (up to 108 992 B used, observed periodically during `acquisition_task` sensor reads) do not threaten memory safety. Importantly, the baseline heap usage *decreased* slightly over 10 minutes (from 107 460 B to 107 432 B), indicating that there are **no measurable memory leaks** over the observation window.

6 File Structure

The repository follows the PlatformIO convention with a `src/` entry point, a `lib/` folder for reusable modules, and a top-level `include/` for headers shared across tasks. Each library module lives in its own subdirectory with an `include/` header folder and a `src/` implementation folder, making cross-compilation units and Doxygen generation straightforward.

Table 7: Repository file structure and module responsibilities.

Path	Purpose
`src/`	Application entry point (`main.c`) and the four FreeRTOS task implementations: `acquisition_task.c`, `audio_task.c`, `comm_task.c`, `system_task.c`. Also contains `task_config.c` (stack/heap logging helpers) and `idf_component.yml` (managed component manifest).
`include/`	Shared task headers (`*_task.h`); `task_config.h` defining stack sizes, task priorities, sensor polling intervals, and MQTT publish cadences.
`lib/config/`	`app_config_template.h` (Wi-Fi, MQTT, QoS, device ID constants) and `esp32_pinout.h` (GPIO/peripheral pin assignments).
`lib/device_state/`	Global `g_device_state` struct and the two FreeRTOS mutexes; all sensor values, timestamps, validity/fault flags, and system flags.
`lib/network/`	Wi-Fi station init (`network_app.c`): NVS, netif, event handlers, reconnect logic, and the connection-ready event group.
`lib/mqtt/`	Four modules: `mqtt_app` (client lifecycle), `mqtt_topic` (topic string builders), `mqtt_payload` (JSON serialisers), `mqtt_publish` (high-level publish API).
`lib/sensor_init/`	Peripheral bring-up: `adc_init`, `i2s_init`, `uart_init` (individual units), and `sensor_init` (orchestrates all).
`lib/` (drivers)	One folder per sensor: `scd40`, `sht41`, `sgp41`, `bmp280`, `as7341`, `pms7003`, `mics5524`, `as312`, `inmp441`. Plus `health` (supervision), `alarm` (event logic), `rgbled` (status LED).
`lib/sgp41/include/`	Also contains Sensirion's BSD-3 Gas Index Algorithm header (`sensirion_gas_index_algorithm.h`).
`server/`	Node.js dashboard: `public/server.js` (MQTT subscriber + Socket.IO relay), `app.js`, `index.html`, `style.css`, `package.json`.
`scripts/`	`update_clangd_toolchain.py`: PlatformIO post-build hook that patches `.clangd` with the correct ESP-IDF include paths for IDE support.
`platformio.ini`	Build configuration: `esp32dev` target, ESP-IDF framework, 4 MB flash, 115200 baud monitor, post-build extra script.
`raport/`	This report (`raport.tex`), the BFH template PDF, and `project_info/` with raw measurement logs.

7 User Manual

> **Prerequisite:** the ESP32 has already been flashed with the firmware. You need a Wi-Fi network and an MQTT broker (e.g. Mosquitto) reachable on the same LAN.

7.1 Step 1 — Configure credentials

Open `lib/config/include/app_config.h` and fill in your values:

```
#define APP_WIFI_SSID        "YourNetworkSSID"
#define APP_WIFI_PASSWORD    "YourNetworkPassword"
#define APP_MQTT_BROKER_URI  "mqtt://192.168.1.10"   // broker IP
#define APP_DEVICE_ID        "esp32_node_01"         // unique ID
```

Leave `APP_MQTT_USERNAME` and `APP_MQTT_PASSWORD` empty if your broker has no authentication. Rebuild and re-flash after editing this file.

7.2 Step 2 — Flash and monitor

From the repository root, run:

```
# Build, upload, and open serial monitor (115200 baud)
pio run -t upload && pio device monitor -b 115200
```

If the serial port is not detected automatically, add `upload_port = /dev/ttyUSB0` (or the appropriate port) to `platformio.ini`.

7.3 Step 3 — Verify boot sequence

Watch the serial output. A successful startup prints, in order:

1. `[MAIN] System booting...`
2. `[MAIN] Sensors initialized successfully`
3. `[MAIN] Wi-Fi connected successfully`
4. `[MAIN] Starting MQTT`
5. `MQTT_APP: MQTT client started`
6. `[MAIN] All tasks started`

If any step fails, the system prints an error and calls `abort()`. Check credentials (`app_config.h`), broker reachability (ping from the same subnet), and GPIO wiring.

7.4 Step 4 — Subscribe to MQTT topics

Use any MQTT client to verify the data stream. Example with `mosquitto_sub`:

```
# All topics for this device
mosquitto_sub -h 192.168.1.10 -t "devices/esp32_node_01/#" -v

# Environment telemetry only
mosquitto_sub -h 192.168.1.10 \
  -t "devices/esp32_node_01/telemetry/environment" -v
```

You should see a new JSON payload roughly every second on the environment topic.

7.5 Step 5 — Start the web dashboard (optional)

```
cd server
npm install                    # first run only
cp .env.example .env           # create local config
```

Edit `.env` and set `MQTT_BROKER_URL=mqtt://192.168.1.10` (and `PORT` if needed), then:

```
npm start
```

Open `http://localhost:3000` (or the configured port) in a browser. Live charts for all sensor channels update automatically via Socket.IO.

7.6 Troubleshooting

- **Wi-Fi fails**: verify SSID/password in `app_config.h`; check that the access point uses WPA2-PSK.
- **MQTT fails**: ping the broker from the LAN; ensure `APP_MQTT_BROKER_URI` starts with `mqtt://` and uses the correct IP and port (default 1883).
- **Sensor missing from payload**: check wiring against `esp32_pinout.h`; the `degraded_mode` flag in `status/system` will be `true` after 45 s if a sensor stops responding.
- **Noise level seems wrong**: the `INMP441_SPL_OFFSET_DB` constant in `lib/inmp441/src/inmp441_w.c` can be adjusted. Compare the reported `noise_db` against a calibrated sound-level meter in a known environment and set `offset = L_meter - L_firmware`.

8 Version control

Table 8: Document revision history.

Ver.	Date	Description	Author
0.1	2026-03-18	Initial RTOS template alignment; basic structure and task overview.	S. / A.
0.2	2026-03-21	Sensor integration notes; first version of requirements table.	S. / A.
0.3	2026-04-06	Architecture refresh: context diagram, acquisition flowchart, TODO metrics.	S. / A.
0.4	2026-04-06	Full English report; layout refresh; MQTT/dashboard/scripts sync with repo.	S. / A.
0.5	2026-04-24	Runtime statistics filled with measured stack/heap values from 10-min logs.	S. / A.
1.0	2026-04-24	Final report: sensor table, analysis boxes, improved design section, UML diagram, extended user manual, troubleshooting guide.	S. / A.